

n8n at the center of your pipelines

Is Airflow too much? Is n8n too little? Let's talk about the elephant in the orchestration room. 🐘



The context

Over the past few months, I've analyzed how data engineers and backend developers automate their processes. The market is split between two polarized extremes: 🚫 Extreme 1: "The Visual Toys" (Zapier, Make, n8n)

Why engineers shift to wpipe

- Pipelines are Python: versionable, testable, reviewable in Git.
- State persisted to SQLite WAL at every step; resume, don't restart.
- <50MB RAM. No broker, no daemon, no metadata server.

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step

@step(name="ingest", retry_count=3)
def ingest(data):
    return {"payload": data["rows"]}

pipe = Pipeline(pipeline_name="from_n8n")
pipe.set_steps([ingest])

# Canonical JSON -> typed, versionable Python steps.
```

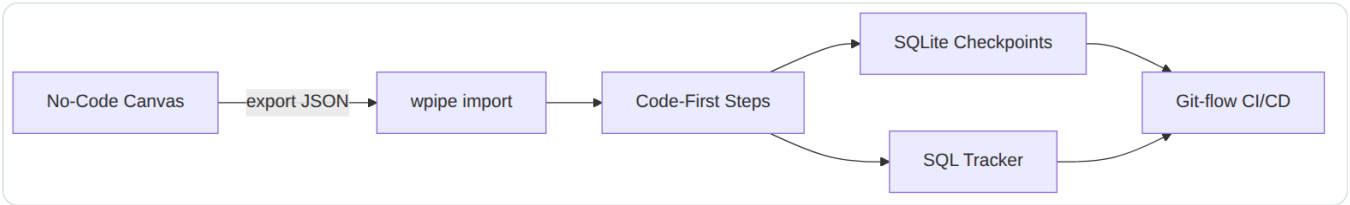
At a glance

Aspect	Legacy / n8n	wpipe
Definition	Config/DSL or canvas	Python code
State	Scattered / none	SQLite WAL checkpoints
Observability	Partial	Tracker SQL + dashboard

Aspect	Legacy / n8n	wpipe
Footprint	Heavy	<50MB RAM

Under the hood

The flow behind the post, as wpipe renders it from code.



+117k downloads
A community of efficiency-first developers.

<50MB RAM
Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL
Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore n8n workflows at wpipe.readthedocs.io.

wpipe · Python-native orchestration with SQLite WAL checkpoints, retries, timeouts, parallelism, and self-documenting flows. +117k downloads.